

十大排序算法

笔记本： 专业课及面试

创建时间： 2021/3/7 15:19

更新时间： 2021/9/23 16:55

作者： 19851621698@163.com

URL: <https://www.cnblogs.com/onepixel/articles/7674659.html>

1.十大经典排序算法

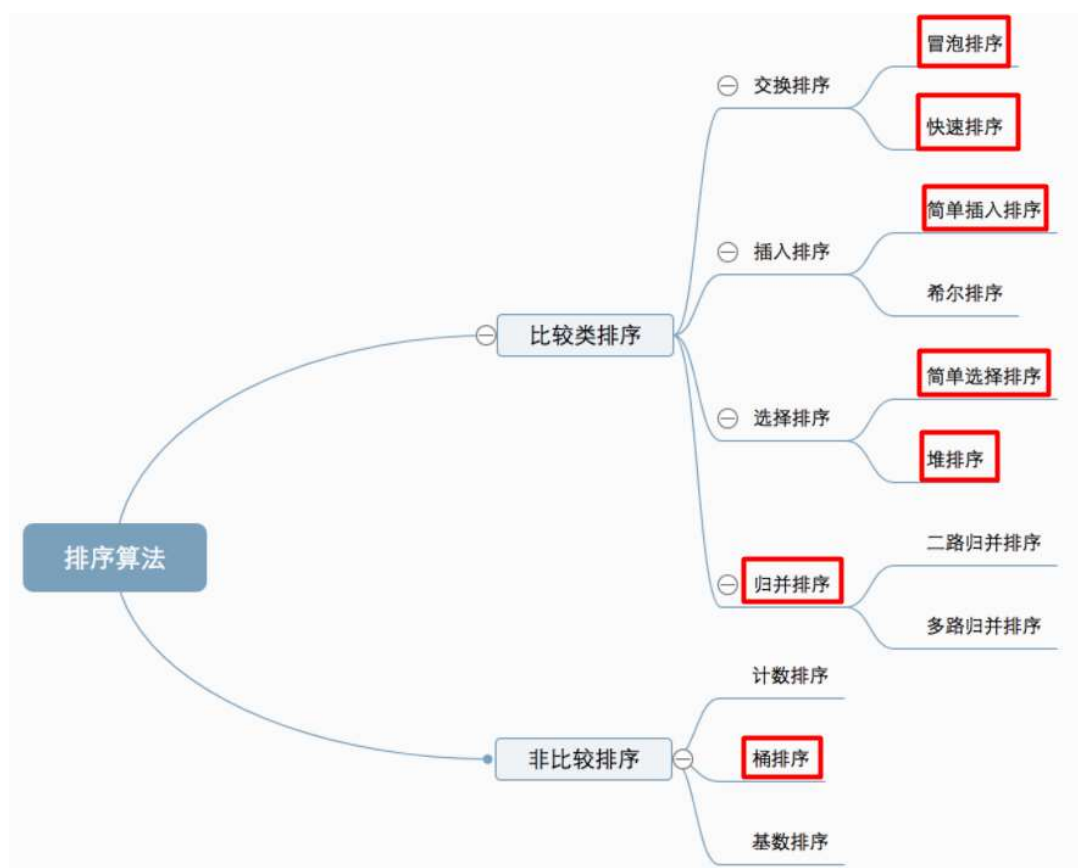
参考资料网址: <https://www.cnblogs.com/onepixel/articles/7674659.html>

0、算法概述

0.1 算法分类

十种常见排序算法可以分为两大类：

- 比较类排序：通过比较来决定元素间的相对次序，由于其时间复杂度不能突破 $O(n\log n)$ ，因此也称为非线性时间比较类排序。
- 非比较类排序：不通过比较来决定元素间的相对次序，它可以突破基于比较排序的时间下限，以线性时间运行，因此也称为线性时间非比较类排序。



0.2 算法复杂度

排序方法	时间复杂度（平均）	时间复杂度（最坏）	时间复杂度（最好）	空间复杂度	稳定性
1. 插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
8. 希尔排序	$O(n^{1.3})$	$O(n^2)$	$O(n)$	$O(1)$	不稳定
2. 选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
4. 堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
3. 冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
5. 快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(n\log_2 n)$	不稳定
4. 归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
计数排序	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	稳定
7. 桶排序	$O(n+k)$	$O(n^2)$	$O(n)$	$O(n+k)$	稳定
基数排序	$O(n*k)$	$O(n*k)$	$O(n*k)$	$O(n+k)$	稳定

0.3 相关概念

- **稳定**：如果a原本在b前面，而a=b，排序之后a仍然在b的前面。
- **不稳定**：如果a原本在b的前面，而a=b，排序之后 a 可能会出现在 b 的后面。
- **时间复杂度**：对排序数据的总的操作次数。反映当n变化时，操作次数呈现什么规律。
- **空间复杂度**：是指算法在计算机内执行时所需存储空间的度量，它也是数据规模n的函数。

1、冒泡排序(Bubble Sort)

冒泡排序是一种简单的排序算法。它重复地走访过要排序的数列，一次比较两个元素，如果它们的顺序错误就把它们交换过来。冒泡排序的本质在于**交换**，即**每次通过交换的方式把当前剩余元素的最大值移动到一端**。

1.1 算法描述

- 比较相邻的元素。如果第一个比第二个大，就交换它们两个；
- 对每一对相邻元素作同样的工作，从开始第一对到结尾的最后一对，这样在最后的元素应该会是最大的数；
- 针对所有的元素重复以上的步骤，除了最后一个；
- 重复步骤1~3，直到排序完成。

1.2 动图演示



1.3 代码实现

```

01. for(int i=1;i<n;i++)//一共比较n-1趟，每一趟都把剩余元素中最大的数移到最右边
02. {
03.     for(int j=0;j<n-i;j++)//第i趟比较n-i次
04.     {
05.         if(a[j]>a[j+1])
06.         {
07.             int temp;
08.             temp=a[j];
09.             a[j]=a[j+1];
10.             a[j+1]=temp;
11.         }
12.     }

```

13. }

1.4 时间复杂度

冒泡排序平均情况下的时间复杂度为 $O(n^2)$ ，是稳定的排序算法。最坏情况下需要进行 $n*(n-1)/2$ 次交换，例如 $a[5]=\{5,4,3,2,1\}$ ，共进行 $4+3+2+1=5*(5-1)/2=10$ 次交换。最好情况下，无需进行交换。平均情况下，需要进行 $n*(n-1)/4$ 次交换。因此时间复杂度为 $O(n^2)$ 。

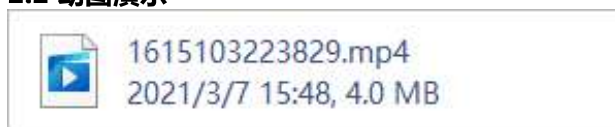
2、选择排序(Selection Sort)

选择排序(Selection-sort)是一种简单直观的排序算法。它的工作原理：首先在未排序序列中找到最小（大）元素，存放到排序序列的起始位置，然后，再从剩余未排序元素中继续寻找最小（大）元素，然后放到已排序序列的末尾。以此类推，直到所有元素均排序完毕。

2.1 算法描述

n 个记录的直接选择排序可经过 $n-1$ 趟直接选择排序得到有序结果。具体算法描述如下：对数组 $a[n]$ ，令 i 从0到 $n-1$ 枚举，进行 $n-1$ 趟操作，每趟从待排序部分 $a[i]-a[n-1]$ 中选择最小的元素，令其与待排序部分的第一个元素 $a[i]$ 进行交换，这样元素 $a[i]$ 就会与前面的有序序列 $a[0]-a[i-1]$ 形成新的有序区间 $a[0]-a[i]$ 。于是在 $n-1$ 趟操作后，所有元素就会是有序的。

2.2 动画演示



2.3 代码实现

```
01. for(int i=0;i<n-1;i++)//共执行n-1趟，第i趟将a[i]到a[n-1]中的最小值与a[i]
    交换
02. {
03.     int min=i;//先假设a[i]是a[i]到a[n-1]区间的最小值,min用来保存最小值的索引
04.     for(int j=i+1;j<n;j++)//找到最小值对应的下标
05.     {
06.         if(a[j]<a[min])
07.         {
08.             min=j;
09.         }
10.     }
11.     int temp=a[i];
12.     a[i]=a[min];
13.     a[min]=temp;
14. }
```

2.4 时间复杂度

简单选择排序的时间复杂度为 $O(n^2)$ ，是不稳定的排序算法。简单选择排序的比较次数与序列的初始排序无关。假设待排序的序列有 n 个元素，则比较次数总是 $n(n-1)/2$ 。进行比较操作的时间复杂度为 $O(n^2)$ ，进行移动操作的时间复杂度为 $O(n)$ 。

3、插入排序(Insertion Sort)

插入排序 (Insertion-Sort) 的算法描述是一种简单直观的排序算法。它的工作原理是通过构建有序序列，对于未排序数据，在已排序序列中从后向前扫描，找到相应位置并插入。

3.1 算法描述

具体算法描述如下：

- 从第一个元素开始，该元素可以认为已经被排序；
- 取出下一个元素，在已经排序的元素序列中从后向前扫描；
- 如果该元素（已排序）大于新元素，将该元素移到下一位置；
- 重复步骤3，直到找到已排序的元素小于或者等于新元素的位置；
- 将新元素插入到该位置后；
- 重复步骤2~5。

3.2 动画演示



1614868220296.mp4
2021/3/7 15:55, 1.5 MB

3.3 代码实现

```
01. for(int i=1;i<n;i++)
02. {
03.     int temp=a[i],j=i;//temp临时存储a[i],j从i开始往前枚举
04.     //结合动画演示,temp比前一个数小,前一个数就往后移一位,直到temp小于前一个数,就把temp插入当前位置
05.     while(j>=1&&temp<a[j-1])
06.     {
07.         a[j]=a[j-1];
08.         j--;
09.     }
10.     a[j]=temp;//插入的位置为j
11. }
```

3.4 时间复杂度

插入排序的时间复杂度为 $O(n^2)$,是稳定的排序算法。对 $a[n]$ 进行升序排序,存在最好和最坏情况。最好情况就是,序列已经是升序排序了,在这种情况下,比较操作仅需 $(n-1)$ 次即可;最坏情况是,序列是降序排序,那么此时需要进行 $n(n-1)/2$ 次。插入排序的赋值操作是比较操作的次数加上 $(n-1)$ 次。平均来说插入排序算法的时间复杂度为 $O(n^2)$ 。因而,插入排序适合数据量小,部分数据有序的情况排序。

4、希尔排序(Shell Sort)

1959年Shell发明,第一个突破 $O(n^2)$ 的排序算法,是简单插入排序的改进版。它与插入排序的不同之处在于,它会优先比较距离较远的元素。希尔排序又叫缩小增量排序。

4.1 算法描述

9.2.3 希尔排序

希尔排序(Shell sort)这个排序方法又称为缩小增量排序(diminishing-increment sort),是1959年由D. L. Shell提出来的。该方法的基本思想是:设待排序元素序列有 n 个元素,首先取一个整数 $gap < n$ 作为间隔,将全部元素分为 gap 个子序列,所有距离为 gap 的元素放在同一个子序列中,在每一个子序列中分别施行直接插入排序。然后缩小间隔 gap ,例如取 $gap = \lceil gap/2 \rceil$,重复上述的子序列划分和排序工作。直到最后取 $gap = 1$,将所有元素放在同一个序列中排序为止。由于开始时 gap 的取值较大,每个子序列中的元素较少,排序速度较快;待到排序的后期, gap 取值逐渐变小,子序列中元素个数逐渐变多,但由于前面工作的基础,大多数元素已基本有序,所以排序速度仍然很快。

下面举例说明。图9.3给出对 $n=6$ 个元素的元素序列进行希尔排序的过程。第1趟取间隔 $gap = \lfloor n/3 \rfloor + 1 = 3$,将整个待排序元素序列划分为间隔为3的3个子序列,分别对其进行直接插入排序;第2趟将间隔缩小为 $gap = \lfloor gap/3 \rfloor + 1 = 2$,将整个元素序列划分为2个间隔为2的子序列,分别对其进行排序;第3趟把间隔缩小为 $gap = \lfloor gap/3 \rfloor + 1 = 1$,对整个序列做直接插入排序,因为此时整个元素序列已经达到基本有序,所以排序速度很快。整个排序的排序码比较次数和元素移动次数少于直接插入排序。

i	(0)	(1)	(2)	(3)	(4)	(5)	gap	i	(0)	(1)	(2)	(3)	(4)	(5)	gap
初始	21	25	49	25*	16	08		2	21	—	08	—	25	—	2
1	21	—	—	25*			3			16	—	25*	—	49	
		25	—	—	16				08	16	21	25*	25	49	
			49	—	—	08		3	08	16	21	25*	25	49	1
	21	16	08	25*	25	49			08	16	21	25*	25	49	

(a) $i=1$ 时希尔排序的过程

(b) $i=2,3$ 时希尔排序的过程

图9.3 希尔排序的过程

为什么该算法会起作用:开始的时候 gap 较大,子序列中的数据较少,所以最开始的时候算法运行较快,随着算法进行, gap 逐渐变小,子序列中元素的个数也就越来越多,所以排序工作可能会变慢,但是由于前面已经完成了部分排序工作,因而在很大程度上减轻了后来的工作量,于是最终总体的排序速度还是比较快的。

4.2 动画演示



1615104516547.mp4
2021/3/7 16:09, 3.3 MB

4.3 代码实现

```
01. const int increment=3;
02. void shellsort(int a[],int len)
03. {
04.     int temp=0;
05.     unsigned gap=len/increment;//对步长gap进行初始化，len为数组的总长度
06.     while(gap)//while gap>=1
07.     {
08.         for(unsigned i=gap;i<len;i++)//分组，在每个子序列内部进行插入排序，i从gap开始是因为插入排序时序列的第一位默认有序，从序列的第二位开始判断
09.         {
10.             temp=a[i];//将当前元素值先存起来方便后面使用
11.             unsigned j=i;
12.             while(j>=gap&&temp<a[j-gap])//依次往前比较，寻找插入位置(可以把gap看为1,就是简单插入排序)
13.             {
14.                 a[j]=a[j-gap];//如果当前元素比前一元素小，将前一元素后移gap位
15.                 j-=gap;//再与前一元素比较
16.             }
17.             a[j]=temp;//直到存储的元素大于前一元素，就把当前元素插入a[j]的位置
18.         }
19.         gap=gap/increment;
20.     }
21. }
```

4.4 时间复杂度

希尔排序适用于规模较大的序列。时间复杂度约为 $O(n^{1.3})$ ，是不稳定的排序算法。



5、归并排序(Merge Sort)

归并排序与快排都使用了two pointers思想，也就是都是使用i、j两个下标对数组进行同时遍历。

归并排序最关键的就是对两个有序序列进行排序，也就是两路归并。

归并排序基于分治法(分而治之)，利用了递归的思想。归并排序将待排序的序列分成两个长度相等的子序列，为每一个子序列排序，然后将它们合并成一个序列。合并两个子序列的过程称为两路归并。

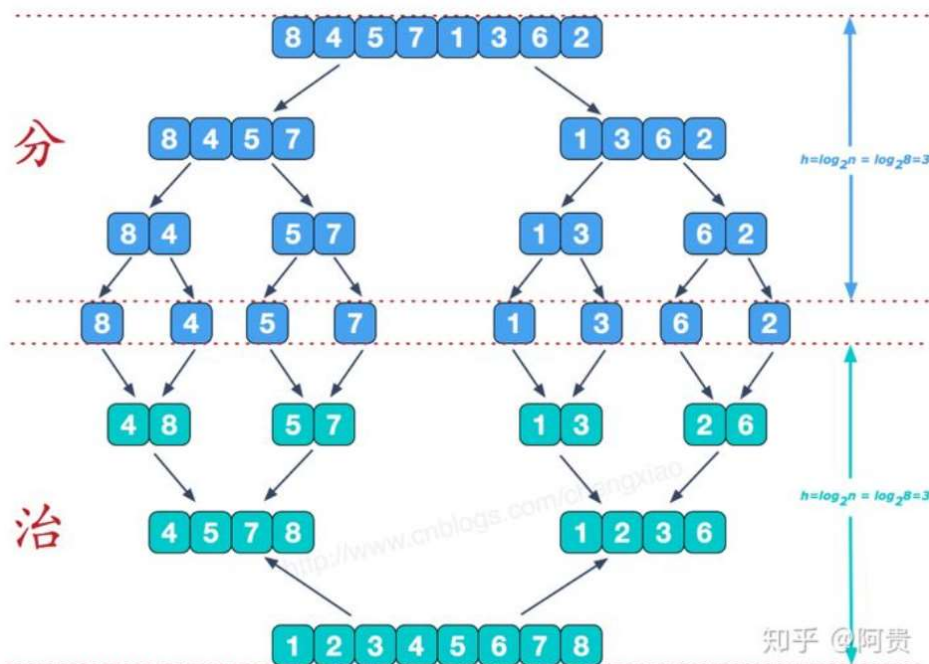
5.1 算法描述

- 把长度为n的输入序列分成两个长度为n/2的子序列；
- 对这两个子序列分别采用归并排序；
- 将两个排序好的子序列合并成一个最终的排序序列。

5.2 动图演示



1615107349328.mp4
2021/3/7 16:56, 4.0 MB



5.3 代码实现

➔ (二路) 归并排序分治算法

■ 算法的基本思想

- **分解**: 将当前待排序的序列 $A[\text{low}], \dots, A[\text{high}]$ 一分为二, 即求分裂点 $\text{mid} = (\text{low} + \text{high}) / 2$;
- **求解**: 递归地对序列 $A[\text{low}], \dots, A[\text{mid}]$ 和 $A[\text{mid}+1], \dots, A[\text{high}]$ 进行归并排序;
- **组合**: 将两个已排序子序列归并为一个有序序列。

■ 递归的终止条件

- 是子序列长度为 1, 因为一个记录自然有序。

```

01. //归并过程
02. void Merge(int a[],int low,int mid, int high)//low为数组的待排序范围的最左边元素的下标, high为数组的待排序范围的最右边元素的下标
03. {
04.     //low为第一有序区的第一个元素, 令i指向第一个元素, mid为第一有序区的最后一个元素
05.     int i=low;
06.     //mid+1为第二有序区的第一个元素, 令j指向第一个元素
07.     int j=mid+1;
08.     int *temp=new int[high-low+1]; //temp数组用来暂存合并的有序序列
09.     int k=0; //k用来索引数组temp
10.     while(i<=mid&& j<=high) //分别从第一个和第二个有序区取值进行比较后存入temp
11.     {
12.         if(a[i]<=a[j]) //将较小的先存入temp数组
13.         {
14.             temp[k++]=a[i++];
15.         }
16.         else
17.         {
18.             temp[k++]=a[j++];
19.         }
20.     }
21.     while(i<=mid) //如果比较完之后, 第1个有序区还有剩余(剩余元素必然大于第2个有序区中所有元素), 则直接复制到t数组中
22.     {
23.         temp[k++]=a[i++];
24.     }
25.     while(j<=high) //果比较完之后, 第2个有序区还有剩余(剩余元素必然大于第1个有序区中所有元素), 则直接复制到t数组中
26.     {

```

```

27.         temp[k++]=a[j++];
28.     }
29.     for(i=low,k=0;i<=high;i++,k++)//将排好序的数组从temp中存回a中的
low~high区间
30.     {
31.         a[i]=temp[k];
32.     }
33.     delete []temp;//释放内存，由于指向的是数组，必须用delete []
34. }
35.
36.
37. //递归
38. void MergeSort(int a[],int low,int high)
39. {
40.     if(low>=high)
41.     {
42.         return;//当子序列长度变为1时，终止递归
43.     }
44.     int mid=low+(high-low)/2;//取得序列中间的元素对应的下标
45.     MergeSort(a,low,mid);//对左半部分递归，进行归并排序
46.     MergeSort(a,mid+1,high);//对右半部分递归，进行归并排序
47.     Merge(a,low,mid,high);
48. }

```

5.4 时间复杂度

归并排序是一种稳定的排序方法。和选择排序一样，归并排序的性能不受输入数据的影响，但表现比选择排序好的多，因为始终都是 $O(n\log n)$ 的时间复杂度。代价（最大问题）是需要额外的内存空间。



5.5 基于递归地归并排序的优化策略

(1) 优化方法1：对小规模的子数组使用插入排序

由于递归会使得小规模问题中方法调用太过频繁，所以可以采用不同的方法处理小规模问题来改进递归算法的性能。因为插入排序非常简单，因此一般来说插入排序在小数组上比归并排序更快。

怎么切换成插入排序呢？只要把作为递归地停止条件的：

```

01. if(low>=high)
02. {
03.     return;//当子序列长度变为1时，终止递归
04. }

```

改成：

```

01. if(high-low<=10)//当数组长度小于10的时候切换到插入排序
02. {
03.     InsertSort(int a,int low,int high);
04.     return;
05. }

```

就可以了。这样的话，这条语句就具有了两个功能：

- 在适当的时候终止递归；
- 当数组长度小于M的时候($high-low \leq M$)，不进行归并排序，改用更快速的插入排序。

(2) 优化方法2：测试待排序序列中左右半边是否已经有序

通过测试待排序序列中左右半边是否已经有序，避免在有序的情况下再次调用合并。

因为 $a[low \dots mid]$ 和 $a[mid \dots high]$ 本来就是有序的，存在 $a[low] < a[low+1] < \dots < a[mid]$ 和 $a[mid+1] < a[mid+2] < \dots < a[high]$ 这两种关系，如果判断出 $a[mid] \leq a[mid+1]$ 的话，我们就认为数组已经是有序的并跳过merge()方法。

```

01. //递归
02. void MergeSort(int a[],int low,int high)
03. {
04.     if(low>=high)
05.     {
06.         return;//当子序列长度变为1时，终止递归
07.     }
08.     int mid=low+(high-low)/2;//取得序列中间的元素对应的下标
09.     MergeSort(a,low,mid);//对左半部分递归，进行归并排序
10.     MergeSort(a,mid+1,high);//对右半部分递归，进行归并排序

```

```

11.     if(a[mid]<=a[mid+1])return;//避免不必要的归并
12.     Merge(a,low,mid,high);
13. }

```



6、快速排序(Quick Sort)

快速排序采用**分治法**进行排序。其**基本思想**是任取待排序元素序列中的某个元素（例如第一个元素作为基准），按照该元素的值大小，将整个序列划分成左右两个子序列：左侧子序列中所有元素都小于基准元素，右侧子序列中所有元素都大于等于基准元素，基准元素则排在这两个子序列中间（这也是该元素最终安放的位置）。

快排之所以被称为“快”排，是因为它在平均时间上最快的，主要原因是硬件方面的，每趟快排需要指定一个“支点”（也就是作为分界点的值），一趟中涉及的所有比较都是与这个“支点”来进行比较的，那么我们可以把这个“支点”放在寄存器里，如此这般，效率自然大大提高。除此之外，快排的高效率与分治思想也是分不开的。

而归并排序则需要一整个数组的额外存储空间，效率不高。

快排采用two pointers思想，使用两个下标对数组进行遍历。一种方法使用pivotpos和i两个下标对数组进行同向遍历，最后pivotpos的值就是基准元素（一般取数组中第一个元素为基准元素）应该存放的位置；另一种方法使用left和right两个下标对数组进行反向遍历，最后left==right时对应的位置就是基准元素应该存放的位置。

6.1 算法描述

- 从数列中挑出一个元素，称为“基准”（pivot）；
- 重新排序数列，所有元素比基准值小的摆放在基准前面，所有元素比基准值大的摆在基准的后面（相同的数可以到任一边）。在这个分区退出之后，该基准就处于数列的中间位置。这个称为分区（partition）操作；
- 递归地（recursive）把小于基准值元素的子数列和大于基准值元素的子数列排序。

6.2 举例演示

- 使用pivotpos和i两个下标对数组进行同向遍历：

例如，图 9.4(a)给出的待排序序列有 6 个元素{21, 25, 49, 25*, 16, 08}，以第一个元素 21 作为基准，对整个元素序列进行划分，一趟下来得到左右两个子序列：{16, 08}和{25*, 49, 25}，左侧子序列所有元素的排序码均小于 21，右侧子序列所有元素的排序码均大于 21。对左侧子序列以 16 为基准，对右侧子序列以 25* 为基准进行同样的划分，就得到最终排序的结果。

i	(0)	(1)	(2)	(3)	(4)	(5)	pivot (基准)
初始序列	[21	25	49	25*	16	08]	21
1	[08	16]	21	[25*	25	49]	08 (左) 25* (右)
2	08	[16]	21	25*	[25	49]	25 (右)
3	08	16	21	25*	25	[49]	

(a) 各趟排序后的结果

i = 1	(0)	(1)	(2)	(3)	(4)	(5)	pivot (基准)
初始序列	[21	25	49	25*	16	08]	21
	↑ pivotpos	↑ i	↑ i	↑ i	↑ i		
循环 4	[21	16	49	25*	25	08]	交换 25 与 16
		↑ pivotpos			↑ i	↑ i	
循环 5	[21	16	08	25*	25	49]	交换 49 与 08
			↑ pivotpos		↑ i	↑ i	
出循环	[08	16]	21	[25*	25	49]	交换 21 与 08

(b) i=1 时快速排序的过程

图 9.4 快速排序的过程

pivotpos最终指向的位置就是基准元素最终放置的位置。

- 使用left和right两个下标对数组进行反向遍历：

的做法，思想就是 two pointers:

① 先将 $A[1]$ 存至某个临时变量 $temp$ ，并令两个下标 $left$ 、 $right$ 分别指向序列首尾（如令 $left = 1$ 、 $right = n$ ）。

② 只要 $right$ 指向的元素 $A[right]$ 大于 $temp$ ，就将 $right$ 不断左移；当某个时候 $A[right] \leq temp$ 时，将元素 $A[right]$ 挪到 $left$ 指向的元素 $A[left]$ 处。

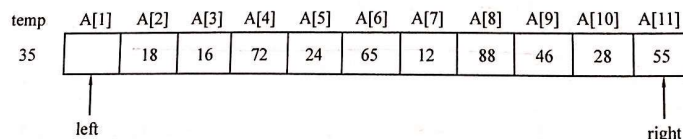
③ 只要 $left$ 指向的元素 $A[left]$ 不超过 $temp$ ，就将 $left$ 不断右移；当某个时候 $A[left] > temp$ 时，将元素 $A[left]$ 挪到 $right$ 指向的元素 $A[right]$ 处。

④ 重复②③，直到 $left$ 与 $right$ 相遇，把 $temp$ （也即原 $A[1]$ ）放到相遇的地方。

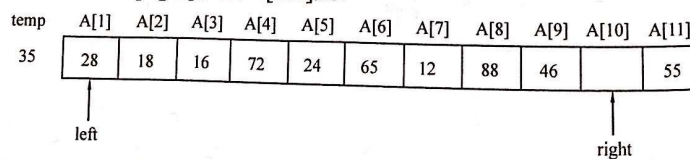
为了使上面的过程更清晰，下面举一个例子。

现有序列 $A[1 \sim 11] = \{35, 18, 16, 72, 24, 65, 12, 88, 46, 28, 55\}$ ，调整元素位置，使得元素 $A[1] = 35$ 的左侧所有元素均不超过 35、右侧所有元素均大于 35。

① 将 $A[1] = 35$ 存到临时变量 $temp$ ，并令下标 $left$ 、 $right$ 指向序列首尾（ $left = 1$ 、 $right = 11$ ）。

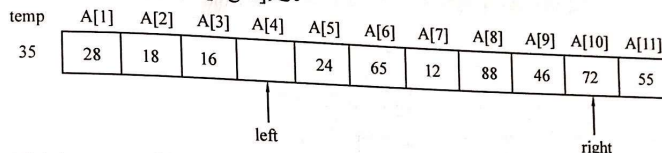


② 只要 $A[right] > 35$ ，就把 $right$ 不断左移。该操作当 $right = 10$ 时满足 $A[right] = 28 < 35$ ， $right$ 停止左移。之后把 $A[right]$ 移至 $A[left]$ 处。

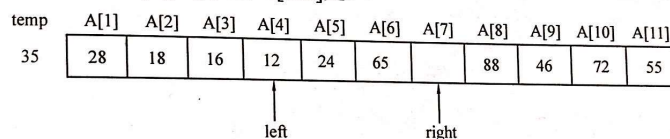


③ 只要 $A[left] \leq 35$ ，就把 $left$ 不断右移。该操作当 $left = 4$ 时满足 $A[left] = 72 > 35$ ， $left$

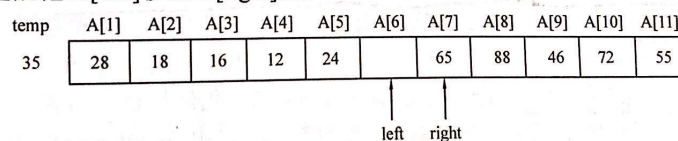
停止右移。之后把 $A[left]$ 移至 $A[right]$ 处。



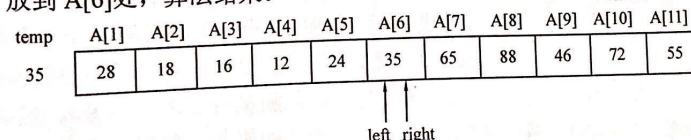
④ 只要 $A[right] > 35$ ，就把 $right$ 不断左移。该操作当 $right = 7$ 时满足 $A[right] = 12 < 35$ ， $right$ 停止左移。之后把 $A[right]$ 移至 $A[left]$ 处。



⑤ 只要 $A[left] \leq 35$ ，就把 $left$ 不断右移。该操作当 $left = 6$ 时满足 $A[left] = 65 > 35$ ， $left$ 停止右移。之后把 $A[left]$ 移至 $A[right]$ 处。



⑥ 只要 $A[right] > 35$ ，就把 $right$ 不断左移。该操作的过程中 $left$ 与 $right$ 在 $A[6]$ 处相遇，将 $temp = 35$ 放到 $A[6]$ 处，算法结束。



6.3 代码实现

- 使用 $pivotpos$ 和 i 两个下标对数组进行同向遍历(数据结构书上的方法):

```
01. //划分，返回序列中基准元素应该放置的位置pivotpos
02. int Partition(int a[],int low,int high)
03. {
04.     int pivot=a[low]; //pivot为基准元素值
05.     int pivotpos=low; //pivotpos指向目前为止小于基准元素的最后一个
    值
06.     for(int i=low+1;i<=high;i++) //对当前序列进行遍历
07.     {
08.         if(a[i]<pivot) //小于基准元素应该放到基准元素左边
```

```

09.         {
10.             pivotpos++;
11.             if(pivotpos!=i)swap(a[pivotpos],a[i]);
12.         }
13.     }
14.     a[low]=a[pivotpos];
15.     a[pivotpos]=pivot;//将基准元素放在它应放的位置
16.     return pivotpos;//返回基准元素的位置，相当于进行划分来完成递归
17. }
18.
19. //递归
20. void QuickSort(int a[],int low,int high)
21. {
22.     //pivot=a[low]是基准元素，排序结束后它的位置在pivotpos
23.     if(low<high)//当元素序列长度大于1时继续递归快排
24.     {
25.         int pivotpos=Partition(a,low,high);//返回该趟基准元素应放的位置
26.         QuickSort(a,low,pivotpos-1);//对左侧子序列进行同样的处理
27.         QuickSort(a,pivotpos+1,high);//对右侧子序列进行同样的处理
28.     }
29. }

```

- 使用left和right两个下标对数组进行反向遍历(《算法笔记》上的方法):

```

01. //划分，返回序列中基准元素应该放置的位置left==right
02. int Partition(int a[],int left,int right)
03. {
04.     int pivot=a[left];//将序列第一个元素记为基准元素
05.     //在left和right两个下标还没有相遇时继续循环
06.     //每次循环先操作right下标，再操作left下标
07.     while(left<right)
08.     {
09.         while(left<right&& a[right]>pivot)//当right指向的元素大于基准元素
            时继续左移
10.         {
11.             right--;
12.         }
13.         a[left]=a[right];//当right指向的元素小于等于基准元素时把right位置的
            元素放在left位置上,此时right位置就空了
14.         while(left<right&& a[left]<=pivot)//当left指向的元素小于等于基准
            元素时继续右移
15.         {
16.             left++;
17.         }
18.         a[right]=a[left];//当left指向的元素大于基准元素时把left位置的元素
            放在right位置上,此时left位置就空了
19.     }
20.     a[left]=pivot;//将基准元素放在它应放的位置
21.     return left;//返回基准元素的位置，相当于进行划分来完成递归
22. }
23.
24. void QuickSort(int a[],int left,int right)
25. {
26.     if(left>=right)return;
27.     int pivotpos=Partition(a,left,right);//pivotpos为基准元素存放的位置
            下标,对数组划分成左右两个子区间
28.     QuickSort(a,left,pivotpos-1);
29.     QuickSort(a,pivotpos+1,right);
30. }

```

6.4 时间复杂度

- 最差情况: 每次我们在划分时，所取的基准总是数组中最小的，因此我们总共会进行 $n-1$ 次划分，且在第 i 次划分时，区间长度为 $n-i+1$ ，需要进行 $n-i$ 次比较。故：

$$C_{max} = \sum_{i=1}^{n-1} n - i = \frac{n(n-1)}{2} = O(n^2)$$

- **最好情况**：每次所取的基准就是该数组的中点，因此一共需要进行n次划分，两个无序子区间需要进行 $2C(n/2)$ 的比较次数。故：

设 $n = 2^k$ 故：

$$C(n) \leq n + 2C\left(\frac{n}{2}\right)$$

$$\leq n + 2\left(\frac{n}{2} + 2C\left(\frac{n}{2^2}\right)\right) = 2n + 4C\left(\frac{n}{2^2}\right)$$

$$\leq kn + nC(1) \leq n \log_2 n + nC(1) = O(n \log_2 n)$$

因此，快排的平均时间复杂度为 $O(n \log n)$ ，是不稳定的排序算法。

6.5 快排的优越性

快排的性能在所有排序算法里面是最好的，数据规模越大快速排序的性能越优。但是当序列中元素接近有序时，会达到最坏时间复杂度 $O(n^2)$ ，产生这种情况的主要原因在于基准元素没有把当前区间划分成两个长度接近的子区间。快排在极端情况下会退化成 $O(n^2)$ 的算法，因此假如在提前得知处理数据可能会出现极端情况(这个极端情况就是在数组有序的情况下)的前提下，可以选择较为稳定的归并排序。

为什么快排是平均性能最好的排序算法？

- 首先，假如a是基准，并且我们已经知道 $a > b$ ，b换到pivotpos位置之后，a,b就不会再次进行比较了，假如再进行比较就是重复。
- 第二，假如a是基准，并且我们已经知道 $b < a < c$ ，换完位置之后，b和c就不会再进行比较了，再进行比较就是冗余。在快排中不会这样重复比较，因为a就是递归排序的基准，因此b,c就只会自己的区间进行排序，不会再出现冗余比较了。

因此，我们可以了解到，快速排序的优越性体现在他没有多余的比较上，对于初学者，我们可以较为简单的认为，快排所需要的指令数会比较少。

7、堆排序(Heap Sort)

事先需要掌握的内容：大顶堆/小顶堆、大顶堆/小顶堆的构造方式。

构造初始化堆(大顶堆)的步骤：

- 如果父亲结点num[a]和它的两个孩子结点num[2a+1], num[2a+2]满足 $\text{num}[a] > \max\{\text{num}[2a+1], \text{num}[2a+2]\}$ ，那么返回；
- 如果不满足堆的性质，那么将父亲结点num[a]和较大孩子结点 $\max\{\text{num}[2a+1], \text{num}[2a+2]\}$ 交换；
- 将原来较大的孩子结点作为父亲结点，重复上述操作，直到孩子结点是叶子结点为止。

堆排序 (Heapsort) 是指利用堆这种数据结构所设计的一种排序算法。堆是一个近似完全二叉树的结构(二叉树在按层序遍历时在遇到第一个NULL指针即作为结尾的二叉树就可以称之为完全二叉树)，并同时满足堆的性质：即子结点的键值或索引总是小于(或者大于)它的父结点。

7.1 算法描述

如果我们建立的堆满足最大堆的条件，则堆的第一个元素 heap[0] 具有最大的排序码，将 heap[0] 与 heap[n-1] 对调，把具有最大排序码的元素交换到最后，再对前面的 n-1 个元素，使用堆的调整算法 siftDown(0, n-2)，重新建立最大堆。结果具有次最大排序码的元素又上浮到堆顶，即 heap[0] 位置，再对调 heap[0] 和 heap[n-2]，并调用 siftDown(0, n-3)，对前 n-2 个元素重新调整，……，如此反复执行，最后得到全部排序好的元素序列。这个算法即堆排序算法，其细节在程序 9.21 中给出。一个执行堆排序的例子如图 9.17 所示。

- 构造初始堆。将给定无序序列构造成为一个大顶堆（一般升序采用大顶堆，降序采用小顶堆）；

- 将堆顶元素与末尾元素交换，将最大元素“沉”到数组末端；
- 重新调整结构，使其满足堆定义，然后继续交换堆顶元素与当前末尾元素，反复执行调整+交换步骤，直到整个序列有序。

7.2 举例演示

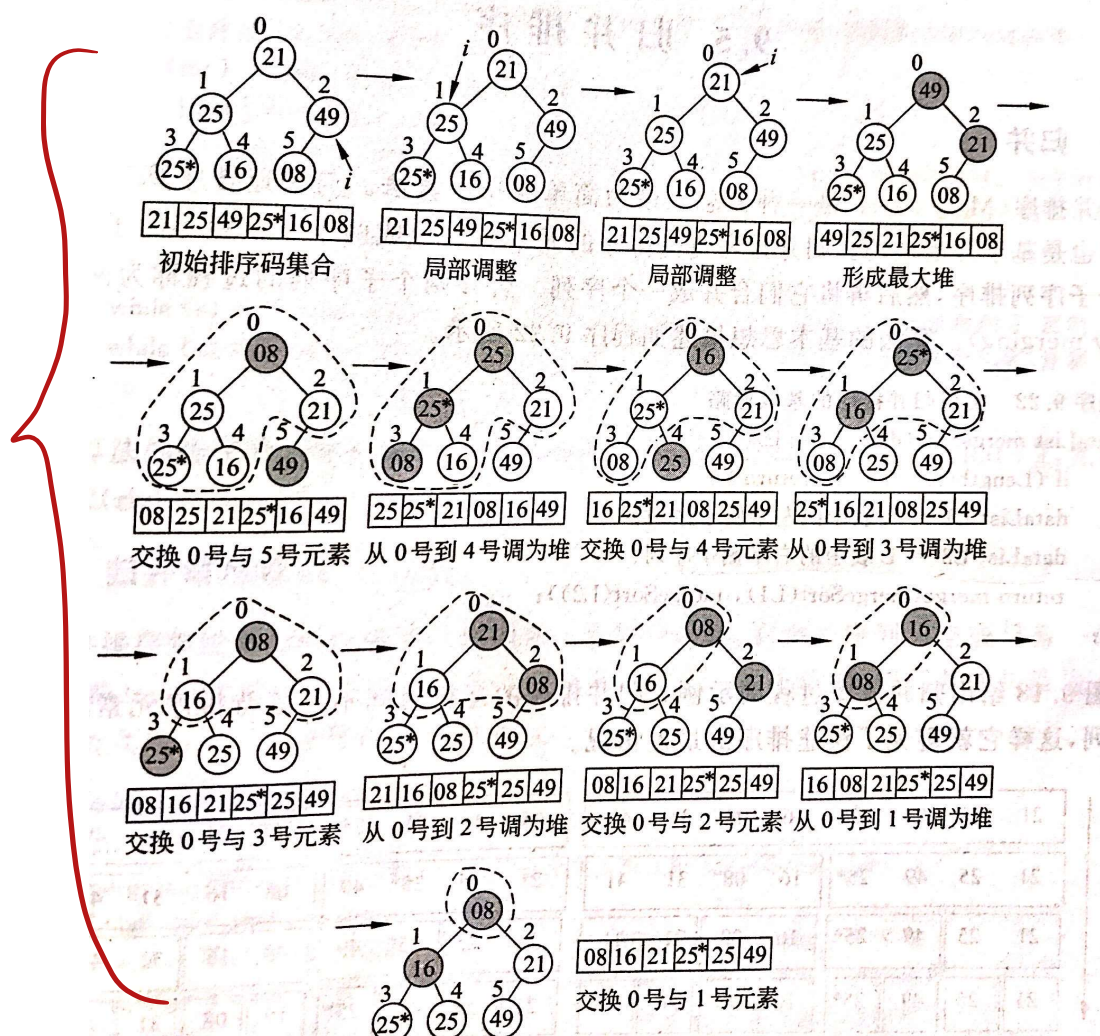


图 9.17 堆排序的示例

7.3 代码实现

```

01. //注:堆实际上是用数组存储的数据结构,只不过使用堆的思路来解决问题
02. //递归方式来构造某棵子树的大顶堆
03. //len是数组arr的长度, index是第一个非叶子节点的下标
04. void adjust(int arr[],int len,int index)
05. {
06.     int left=2*index+1;//index的左子节点
07.     int right=2*index+2;//index的右子节点
08.
09.     int maxIdx=index;//maxIdx存储父节点和左右子节点中最大值对应的下标
10.     if(left<len&&arr[left]>arr[maxIdx]) maxIdx=left;//如果左子节点大于
        父节点,则交换
11.     if(right<len&&arr[right]>arr[maxIdx]) maxIdx=right; //如果右子节点
        大于父节点,则交换
12.
13.     if(maxIdx!=index)//如果最大值非父节点,需要进行交换
14.     {
15.         swap(arr[maxIdx],arr[index]);
16.         adjust(arr,len,maxIdx);//以交换的那个节点为下一棵子树的根节点,
        构造大顶堆
17.     }
18. }
19.
20. //堆排序
21. void HeapSort(int arr[],int len)

```



```

22. {
23.     //先构造大顶堆(从最后一个非叶子节点向上)
24.     for(int i=len/2-1;i>=0;i--)
25.     {
26.         adjust(arr,len,i);
27.     }
28.
29.     //反复进行交换和调整大顶堆
30.     //第一次调整完还剩len-1个元素未排序，第二次调整完还剩len-2个元素未排序
    .....第i次调整完，还剩len-i个元素未排序
31.     for(int i=len-1;i>=1;i--)
32.     {
33.         swap(arr[0],arr[i]); //将当前最大的(也就是根节点arr[0])放置到数组
        末尾
34.         adjust(arr,i,0); //再对剩余的堆进行调整，将下一个次大的数调整到a[0]
        的位置
35.     }
36. }

```



7.4 时间复杂度

初始化堆也就是刚开始构造大顶堆的时间复杂度为 $O(n)$ 。

初始化堆的时候，对于每个非叶子结点，都要调用上述函数，将它与它的孩子结点进行比较和交换，顺序是从后往前。

假设元素个数为 n ，则堆的高度为 $k = \log_2(n+1) \approx \log_2 n$ ，非叶子结点的个数为 $2^{k-1}-1$ 。

假设每个非叶子结点都需要进行调整，则第 i 层的非叶子结点需要的操作次数为 $k-i$ 。

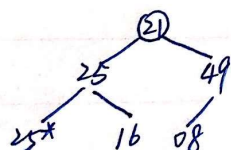
共 $k-1$ 层非叶子结点，总的操作次数为：

$$\sum_{i=1}^{k-1} (k-i) \times 2^{i-1}$$
 第 i 层共 2^{i-1} 个结点，则第 i 层所有结点所做的操作为 $(k-i) \times 2^{i-1}$ 。

化简上式得，上式 $= 2^k - k + 1$ ，将 $k = \log_2(n+1) \approx \log_2 n$ 代入，得上式 $= n - \log_2 n + 1$ 。

所以，初始化堆的复杂度为 $O(n)$ 。

举例=



元素个数为6，堆高度为 $k = \log_2 7 \approx 3$

非叶子结点个数为 $2^{3-1}-1=3$

第1层的非叶子结点的操作次数为2次(比较交换)

.....

共2层非叶子结点，总操作次数为 $\sum_{i=1}^2 (2-i) \times 2^{i-1}$ 。

调整堆的时间复杂度为 $O(n \log n)$ 。

假设根节点和排在最后的序号为 m 的叶子结点交换，并进行调整，那么调整的操作次数 = 原来 m 结点所在的层数 = 堆的高度 (因为 m 结点在堆的最后) = $\log m$ 。

共 n 个结点，调整的总操作次数为：

$$\sum_{m=1}^{n-1} \log m$$

化简可得，上式 $= \log(n-1)! \approx n \cdot \log n$ 。

因此，总体的时间复杂度为 $O(n \cdot \log n)$ ，是不稳定的排序算法。

8、桶排(Bucket Sort)

桶排序是最快最简单的排序。

假设数据服从均匀分布，将数据分到有限数量的桶里，每个桶再分别进行排序(有可能再使用别的排序算法或是以递归的方式继续使用桶排序进行排序)。

8.1 算法描述

- 根据待排序集合中最大元素和最小元素的差值范围和映射规则，确定申请的桶个数；
- 遍历待排序集合，将每一个元素移动到对应的桶中；
- 对每一个不是空的桶进行排序，可以使用其它排序方法，也可以递归使用桶排序；

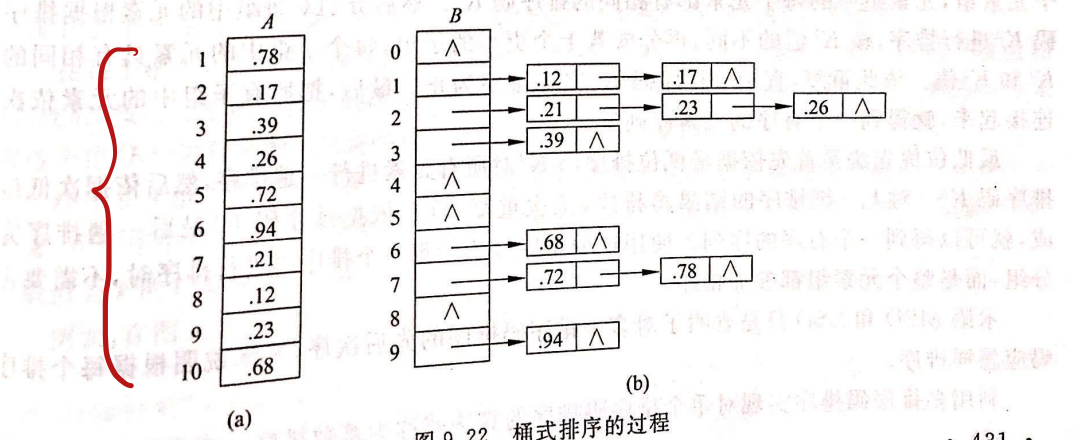
- 从不是空的桶里把排好序的数据拼接起来。

桶排序过程存在两个关键环节：

- 元素值域的划分，也就是元素到桶的映射规则。映射规则需要根据待排序集合的元素分布特性进行选择，若规则设计的过于模糊、宽泛，则可能导致待排序集合中所有元素全部映射到一个桶上，则桶排序向比较性质排序算法演变。若映射规则设计的过于具体、严苛，则可能导致待排序集合中每一个元素值映射到一个桶上，则桶排序向计数排序方式演化。
- 排序算法的选择，从待排序集合中元素映射到各个桶上的过程，并不存在元素的比较和交换操作，在对各个桶中元素进行排序时，可以自主选择合适的排序算法，桶排序算法的复杂度和稳定性，都根据选择的排序算法不同而不同。

8.2 举例演示

下面用一个例子说明桶式排序(bucket sort)的过程。如图 9. 22(a)所示, A 是待排序的元素序列。在图 9. 22(b)中, B 是一系列等长划分的半开半闭区间：[0, 0. 1), [0. 1, 0. 2), ..., [0. 9, 1), 这就是一系列的桶。待排序的元素依次按照自己所属的区间被放入桶中。每个桶中元素序列的规模大为缩小, 只需对每个桶中的子序列进行排序, 然后按桶的编号依次输出就可以得到整个有序的元素序列。



8.3 代码实现

```
01. #include <stdio.h>
02. int main()
03. {
04.     int book[1001], i, j, t, n;
05.     for(i=0; i<=1000; i++)
06.         book[i]=0;
07.     scanf("%d", &n); //输入一个数n, 表示接下来有n个数
08.     for(i=1; i<=n; i++) //循环读入n个数, 并进行桶排序
09.     {
10.         scanf("%d", &t); //把每一个数读到变量t中
11.         book[t]++; //进行计数, 对编号为t的桶放一个小旗子
12.     }
13.     for(i=1000; i>=0; i--) //依次判断编号1000~0的桶
14.         for(j=1; j<=book[i]; j++) //出现了几次就将桶的编号打印几次
15.             printf("%d ", i);
16.     // getchar();
17.     // getchar();
18.     return 0;
19. }
```

8.4 时间复杂度

将元素分配至各个桶中的操作的时间复杂性为O(n)。假设我们对每个桶中的元素子序列采用直接插入排序, 那么每个子序列排序的时间复杂度为O(n^2), 所以整个桶排序的时间复杂度为：

$$T(n) = O(n) + \sum_{i=1}^n O(n^2)$$

可以证明整个桶式排序的时间复杂度为O(n) 是稳定的排序算法。因此, 对于均匀分布的元素序列, 桶式排序的时间开销是线性增长的。

